

MATRIZES ESPARSAS

Mapa de Dependências com Lista Encadeada

1. Descrição do Problema

Um sistema acadêmico precisa representar as dependências entre disciplinas de um curso: qual disciplina é pré-requisito de qual. A relação entre disciplinas pode ser modelada como uma matriz disciplina $\times$ disciplina onde o valor 1 na posição (i, j) indica que a disciplina i depende da disciplina j para ser cursada.

A matriz resultante é naturalmente esparsa: em um currículo real, cada disciplina depende de poucas outras (tipicamente 0 a 3), tornando a grande maioria das entradas igual a zero. Armazenar explicitamente todos esses zeros seria um desperdício de memória — para $n = 100$ disciplinas, seriam 10.000 entradas, das quais talvez menos de 300 seriam não-nulas.

O problema exige, portanto, uma estrutura que armazene apenas as dependências existentes, suporte inserção ordenada, permita verificação eficiente, e forneça operações de navegação pelo grafo de pré-requisitos.

1.1 Representação como Grafo Direcionado

O conjunto de disciplinas e suas dependências forma um Grafo Acíclico Dirigido (DAG — Directed Acyclic Graph). Cada disciplina é um vértice; cada dependência é uma aresta dirigida de i para j (i depende de j). A exigência de ser acíclico é inerente ao problema: se A depende de B que depende de A , nenhuma das duas poderia ser cursada — o que é inválido em qualquer currículo real.

2. Estrutura de Dados Escolhida

2.1 Lista Encadeada de Nós Esparsos

A estrutura escolhida é uma lista encadeada simples e ordenada de nós, onde cada nó armazena um par (linha, coluna) representando uma dependência existente. A lista é mantida em ordem lexicográfica crescente pelo par (linha, coluna) a cada inserção.

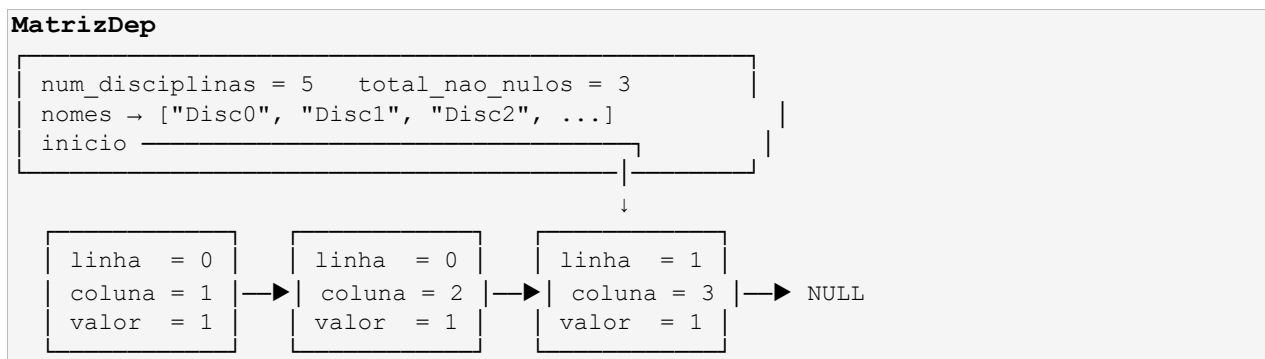
As structs usadas são:

```
typedef struct No {
    int linha;      /* índice da disciplina que depende */
    int coluna;     /* índice do pré-requisito          */
    int valor;      /* sempre 1 (dependência existe) */
    struct No *prox;
} No;

typedef struct {
    int num_disciplinas;
    int total_nao_nulos;
    No *inicio; /* lista ordenada por (linha, coluna) */
    char **nomes; /* vetor de strings dinâmico: nomes[i] */
} MatrizDep;
```

2.2 Diagrama da Estrutura

O diagrama abaixo representa a organização interna da estrutura para uma matriz com as dependências $0 \rightarrow 1$, $0 \rightarrow 2$ e $1 \rightarrow 3$ cadastradas:



2.3 Por que a Lista Encadeada é Adequada

A lista encadeada ordenada é especialmente adequada para este problema pelos seguintes motivos:

- Esparsidade: apenas dependências existentes ocupam memória. Para $n = 50$ disciplinas com 80 dependências, usa-se 80 nós em vez de 2.500 entradas.
- Inserção e remoção $O(k)$ onde k é o número de elementos na lista — eficiente para grafos esparsos reais.
- Percurso por linha eficiente: como a lista está ordenada por (linha, coluna), listar_prerequisitos percorre apenas os nós da linha desejada e para ao ultrapassá-la.
- Flexibilidade de tamanho: não exige pré-alocação de n^2 entradas nem redimensionamento.
- Comparação com alternativas: a matriz densa $n \times n$ seria mais rápida em acesso $O(1)$, mas desperdiçaria $O(n^2)$ memória. O CSR seria mais eficiente em leitura por linha, mas mais complexo para inserção/remoção dinâmica.

3. Implementação

3.1 Organização do Código

O código foi organizado em um único arquivo C (matriz_dep.c) com 627 linhas, dividido nas seguintes seções por blocos de comentários:

- Estruturas de dados (No, MatrizDep)
- Funções auxiliares (exibir_nome)
- Operações obrigatórias (8 funções)
- Operações extras — DFS e Ordenação Topológica (3 funções)
- Gerenciamento de nomes (definir_nome)
- Menu interativo e main

3.2 Gerenciamento de Memória

Toda alocação dinâmica usa malloc/calloc com verificação explícita de retorno NULL antes de prosseguir. A liberação segue o princípio de liberar na ordem inversa da alocação:

```

/* criar_matriz: aloca a struct e o vetor de ponteiros de nomes */ MatrizDep
*m = (MatrizDep *)malloc(sizeof(MatrizDep));
if (!m) { fprintf(stderr, "Erro: malloc falhou\n"); return NULL; }
m->nomes = (char **)calloc(n, sizeof(char *)); /* calloc zera → NULL seguro */

/* inserir_dependencia: aloca cada nó individualmente */ No *novo =
(No *)malloc(sizeof(No)); if (!novo) { fprintf(stderr, "Erro:
malloc falhou\n"); return -1; }

/* liberar_matriz: percorre lista e libera cada nó, depois os nomes
*/ No *cur = m->inicio; while (cur) { No *prox = cur->prox; free(cur);
cur = prox; } for (int i = 0; i < m->num_disciplinas; i++) free(m-
>nomes[i]); free(m->nomes); free(m);

```

3.3 Manutenção da Ordenação

A inserção mantém a lista sempre ordenada por (linha, coluna) usando comparação lexicográfica. O percurso na inserção também detecta duplicatas e para cedo quando ultrapassou a posição de inserção:

```

No *ant = NULL, *cur = m->inicio; while
(cur) {
    if (cur->linha == origem && cur->coluna == destino) {
        printf("Aviso: dependencia ja existe.\n"); return -1;    }
    /* comparação lexicográfica: para quando ultrapassou */
    if (cur->linha > origem || (cur->linha == origem && cur->coluna >
destino))        break;        ant = cur; cur = cur->prox;
}
/* inserção entre ant e cur */ novo->prox = cur;
if (ant) ant->prox = novo; else m->inicio = novo;

```

3.4 Operação: Transposta

A transposição cria uma nova MatrizDep e reinsere cada dependência com linha e coluna trocadas. Como `inserir_dependencia` já garante a ordem, a transposição é correta mesmo sem reordenar explicitamente:

```

MatrizDep *transpor(const MatrizDep *m) {
    MatrizDep *t = criar_matriz(m->num_disciplinas);
    No *cur = m->inicio;    while (cur) {
        inserir_dependencia(t, cur->coluna, cur->linha); /* troca */
        cur = cur->prox;
    }
    return t; /* caller deve chamar liberar_matriz(t) */
}

```

3.5 Operação: pode_cursar

Verifica se todos os pré-requisitos de uma disciplina estão na lista de disciplinas já cursadas fornecida pelo usuário. Percorre apenas a sublista da linha correspondente:

```

int pode_cursar(const MatrizDep *m, int disciplina,
const int *cursadas, int num_cursadas) {
    No *cur = m->inicio;    while
(cur) {                    if (cur->linha ==
disciplina) {              int
encontrado = 0;
        for (int k = 0; k < num_cursadas; k++)                if
(cursadas[k] == cur->coluna) { encontrado = 1; break; }          if
(!encontrado) return 0; /* pré-req não satisfeito */
    }
    if (cur->linha > disciplina) break;
cur = cur->prox;
}
return 1; }

```

4. Funcionalidades Extras — DFS e Ordenação Topológica

4.1 Motivação

O enunciado exige detecção de dependências circulares diretas (A depende de B e B depende de A). Esta implementação vai além: detecta ciclos de qualquer comprimento ($A \rightarrow B \rightarrow C \rightarrow A$) usando DFS (Busca em Profundidade) completa. Adicionalmente, quando o grafo é acíclico, gera a ordenação topológica — uma sequência válida de cursagem de todas as disciplinas respeitando todos os pré-requisitos.

Estas funcionalidades são relevantes porque: (1) ciclos de comprimento maior que 2 são igualmente inválidos e passariam despercebidos pela verificação simples; (2) a ordenação topológica é uma aplicação direta e útil do grafo de pré-requisitos, respondendo à pergunta "em qual ordem devo cursar as disciplinas?"

4.2 Algoritmo DFS com Coloração de Vértices

A DFS usa três estados (cores) para cada vértice, clássicos na detecção de ciclos em grafos:

- 0 — Não visitado (branco): vértice ainda não descoberto
- 1 — Em pilha (cinza): vértice em visita ativa na recursão atual
- 2 — Concluído (preto): todos os vizinhos já foram explorados

Um ciclo é detectado quando, durante a DFS, encontramos uma aresta para um vértice cinza — ou seja, um vértice que já está na pilha de chamadas atual. Isso significa que existe um caminho de volta ao nó já em processamento.

```

        if (dfs_visita(m, viz, estado, ...)) return 1;
    }
    if (cur->linha > v) break;
cur = cur->prox;
}
estado[v] = 2; /* marca como concluído */      if (ordem_topo)
ordem_topo[*idx_topo--] = v; /* coleta para topológica */      return 0; }

/* estado[v]: 0=não visitado, 1=em pilha (cinza), 2=concluído */ static
int dfs_visita(const MatrizDep *m, int v, int *estado, ...) {
    estado[v] = 1; /* marca como "em pilha" */
    No *cur = m->inicio;    while (cur) {        if (cur->linha == v) {
int viz = cur->coluna;        if (estado[viz] == 1) return 1; /*
aresta para cinza = CICLO */        if (estado[viz] == 0)

```

4.3 Ordenação Topológica

A ordenação topológica é obtida como subproduto da DFS: ao concluir um vértice (colorir de preto), ele é inserido no final de um vetor resultado preenchido da direita para a esquerda. Ao final, o vetor contém os vértices em ordem topológica (do mais básico ao mais avançado).

A lógica é: uma disciplina só é "concluída" na DFS depois que todos os seus pré-requisitos já foram concluídos. Portanto, a ordem de conclusão, lida de trás para frente, é uma ordem topológica válida.

Exemplo com 5 disciplinas (0→1, 0→2, 1→3, 2→3, 3→4):

```

Entrada: 0→1, 0→2, 1→3, 2→3, 3→4 Ordem
topológica gerada: 0, 2, 1, 3, 4
Interpretação: cursar Disc0, depois Disc2 ou Disc1 (ambas só dependem de
Disc0),                depois Disc3 (depende de 1 e 2, ambas cursadas),
depois Disc4.

```

5. Execução e Testes

5.1 Teste 1 — Operações Básicas

Foram criadas 5 disciplinas (Disc0–Disc4) com as dependências: Disc0→Disc1, Disc0→Disc2, Disc1→Disc3, Disc2→Disc3, Disc3→Disc4. A matriz exibida confirma a ordenação correta dos nós na lista:

Matriz de dependências (5x5):

	0	1	2	3	4
0	0	1	1	0	0
1	0	0	0	1	0
2	0	0	0	1	0
3	0	0	0	0	1
4	0	0	0	0	0

Total de dependências cadastradas: 5

5.2 Teste 2 — Detecção de Ciclo e Ordenação Topológica

Sem ciclo: o sistema detecta corretamente que o grafo é acíclico e gera a ordem de cursagem. Com ciclo (adicionando $\text{Disc4} \rightarrow \text{Disc0}$): o ciclo é identificado e a ordenação topológica informa que é impossível.

Sem ciclo	Com ciclo ($4 \rightarrow 0$ adicionado)
-----------	---

```
Nenhum ciclo detectado. O grafo e
aciclico.
```

```
Ordem topologica:
```

```
1. [0] Disc0
2. [2] Disc2 3.
[1] Disc1 4.
[3] Disc3
5. [4] Disc4
```

```
CICLO DETECTADO!
```

```
Caminho: [0] -> [4] -> [0]
```

```
Ordenacao topologica
```

```
impossivel: grafo possui ciclo.
```

5.3 Teste 3 — Casos Limite

- Inserção duplicada: ao tentar inserir (0→1) duas vezes, o sistema exibe aviso e rejeita a segunda inserção.
- Índices inválidos: inserir dependência com índice -1 ou fora do intervalo exibe mensagem de erro e retorna -1.
- Disciplina depende de si mesma: inserir (2→2) é rejeitado com mensagem específica.
- Remoção inexistente: remover dependência não cadastrada exibe aviso e retorna -1.
- Pode_cursar sem pré-requisitos: disciplina sem nenhum pré-requisito retorna "pode ser cursada" com lista vazia.
- Ciclo direto (A→B e B→A): detectado corretamente pela DFS como ciclo de comprimento 2.

5.4 Teste 4 — Transposição

A transposta de uma matriz de dependências tem significado semântico: se $M[i][j] = 1$ significa "i depende de j", então a transposta $M[j][i] = 1$ significa "j é pré-requisito de i" — ou equivalentemente, "j habilita i". A função transpor foi testada confirmando a inversão correta de todas as arestas.

6. Discussão e Análise Crítica

6.1 Dificuldades Encontradas

Gerenciamento de ponteiro durante remoção

A remoção de um nó intermediário exige manter um ponteiro anterior (ant). O caso especial é quando o nó a remover é o primeiro da lista (ant == NULL), onde m->inicio deve ser atualizado diretamente. Confundir esses dois casos causou falhas de segmentação durante o desenvolvimento.

Ordenação da lista na transposição

Na transposição, trocar linha e coluna de todos os nós não garante que a lista permaneça ordenada. A solução adotada foi recriar a lista do zero usando inserir_dependencia, que já garante a ordenação. Uma alternativa seria reordenar in-place com um algoritmo de ordenação de lista, mas isso seria mais complexo e desnecessário dado o tamanho típico dos grafos de pré-requisitos.

DFS recursiva e profundidade da pilha

Para grafos muito grandes (centenas de disciplinas com longas cadeias de dependência), a DFS recursiva pode estourar a pilha de chamadas do SO. Para o contexto do problema (currículos acadêmicos com dezenas de disciplinas), isso não é um risco prático. Em produção industrial, uma DFS iterativa com pilha explícita seria preferível.

6.2 Limitações da Solução

- A função `imprimir_matriz` tem complexidade $O(n^2 \times k)$ onde k é o tamanho médio da lista, pois para cada célula da grade faz uma busca na lista. Para n grande, isso pode ser lento. Uma otimização seria usar a ordenação da lista para imprimir linha por linha sem buscas repetidas.
- A busca `listar_dependentes` percorre toda a lista ($O(\text{total_nao_nulos})$) porque a lista não está indexada por coluna. Para grafos densos, isso pode ser um gargalo. Uma lista secundária ordenada por coluna resolveria ao custo de maior complexidade.
- Os nomes de disciplinas são strings independentes alocadas com `malloc`. Em versões futuras, um hash ou tabela de símbolos seria mais adequado para garantir unicidade.
- Não há persistência: ao encerrar o programa, todos os dados são perdidos. Uma extensão natural seria serialização para arquivo texto.

6.3 Complexidade das Operações

Operação	Complexidade	Observação
<code>inserir_dependencia</code>	$O(n)$	n = nós na lista; para na posição certa
<code>remover_dependencia</code>	$O(n)$	Para quando ultrapassa a posição
<code>tem_dependencia</code>	$O(n)$	Busca binária possível, mas desnecessária
<code>listar_prerequisitos</code>	$O(n_i)$	n_i = nós da linha i (para early)
<code>listar_dependentes</code>	$O(n)$	Percorre toda a lista (sem índice por coluna)
<code>transpor</code>	$O(n \log n)$	n reinserções ordenadas
<code>detectar_ciclo</code>	$O(V + E)$	DFS clássica: V vértices, E arestas
<code>ordenacao_topologica</code>	$O(V + E)$	DFS clássica + coleta de conclusão

7. Conclusão

O trabalho implementou com sucesso um sistema completo de gerenciamento de dependências entre disciplinas usando matriz esparsa representada por lista encadeada ordenada. A escolha da estrutura foi adequada ao problema: o grafo de pré-requisitos é naturalmente esparsa, e a lista encadeada oferece inserção/remoção eficientes sem desperdício de memória.

Além das operações obrigatórias, o grupo implementou detecção de ciclos por DFS completa e ordenação topológica — funcionalidades que vão além do enunciado e demonstram compreensão do grafo subjacente. A detecção de ciclos diretos exigida pelo enunciado é um caso particular da DFS implementada; a abordagem mais geral é ao mesmo tempo mais correta e não mais complexa de implementar.

O maior aprendizado técnico foi o gerenciamento de ponteiros em operações de lista: a inserção ordenada exige atenção ao caso de inserção no início (`ant == NULL`), e a remoção exige cuidado para não perder o encadeamento. A DFS recursiva, por sua vez, revelou a elegância da coloração de vértices como mecanismo de detecção de ciclos — um conceito transferível para qualquer linguagem e qualquer representação de grafo.

8. Referências Bibliográficas

- CORMEN, T. H. et al. Algoritmos: Teoria e Prática. 3. ed. Rio de Janeiro: Elsevier, 2012. Cap. 22: Elementary Graph Algorithms (DFS, Topological Sort).
- SEDGEWICK, R.; WAYNE, K. Algorithms. 4. ed. Addison-Wesley, 2011. Cap. 4: Graphs.
- KERNIGHAN, B. W.; RITCHIE, D. M. The C Programming Language. 2. ed. Prentice Hall, 1988.
- TENENBAUM, A. M.; LANGSAM, Y.; AUGENSTEIN, M. J. Estruturas de Dados Usando C. São Paulo: Makron Books, 1995. Cap. 4: Linked Lists.
- ISO/IEC 9899:1999 — Programming languages — C. International Organization for Standardization, 1999 (padrão C99).